Extended Abstract

Motivation LLMs have exploded in popularity over the last few years. The rise of Deepseek showed that model training could be bootstrapped with GRPO to compete with state-of-the-art models like those of OpenAI. We therefore wanted to test this to see if with small LLMs, significant improvements in model quality post-GRPO fine-tuning could still be seen.

Method We finetune the Qwen2.5-Coder-0.5B model with Group Relative Policy Optimisation (GRPO), a variant of the the popular reinforcement learning algorithm Proximal Policy Optimisation (PPO), instead of using a learned value network it instead gets the baseline from other completions of the same prompt. Our code swaps human labellers with fully-automated Rust compiler rubrics. This cuts GPU memory usage roughly in half. GRPO generates multiple outputs per query; for each output, GRPO calculates a reward for how well that output answers the query using different reward functions; then, it calculates the advantage for each output by looking at the mean and standard deviation of all the rewards. Lastly, a KL-Divergence term is used to ensure that the trained policy model does not stray too far from the reference model. We select effective reward signals to improve the quality of the code generated – first, starting with the minimal extrinsic signals such as build, clippy and test; then, developing the signals to include structural and intrinsic rewards such as code-block presence and assert coverage.

Implementation First, for dataset preparation, we start by sampling 15,000 Python prompts from the AceCoder-87K dataset to be used to for the training and evaluation of our model. We translate all these prompts into Rust using GPT o4-mini. For our uses, we used 500 of the prompts for evaluation and the remaining 14,500 for training the model. Second, when training the model, for every prompt the model generates $K = 4$ outputs. Again, we selected different specific reward signals for our reward rubric, which is then used to determine the rewards for each training step and output within the epoch of GRPO. Starting with Build Pass, Clippy Pass, Test Pass and *Rust* Code Block, we developed into a 7-signal model with Cfg Test Rewards and Assert Coverage along with a non-empty test bodies. We also inject LoRA adapters to help finetune the weights.

Results We collect our results by running the models on the 500 prompt evaluation set from our translated AceCoder dataset. The baseline model compiled and linted a nearly a quarter of the queries with Build and Clippy pass rates of 24.8%, however, it only passed the unit tests on 1.4% of cases. After its epoch of GPRO finetuning, the 4-signal reward model (rewarded for Build, Clippy and Test Passes along with Rust code fence) increased the Build and Clippy pass rates to 36%, but fell to 0.2% on genuine test and full pass rates due to reward hacking and cheating the tests. Finally, after adding the three structural and intrinsic incentives in the 7-signal model, the Build and Clippy pass rates landed at 30%, whilst the test completion and full pass rates jumped to 7.2%. This confirmed that the 7-signal reward rubric better optimised for genuine unit-test success.

Discussion From our results, it is clear that the formation of the reward signals are key to the effectiveness of GRPO – as such, they must be designed in a way that prevents reward hacking where the model optimises for the reward function in unproductive ways. It would also be fruitful to explore different mixtures of reward signals with different weights to see just how effective GRPO's finetuning and reward signals can be.

Conclusion Our study shows that a 0.5-B parameter LM, finetuned with a GRPO loop can significantly increase the quality of Rust code generation using compiler feedback and an A100. The differences in the results from the 4-signal and 7-signal reward rubrics supports the idea that the formulation of the reward rubric and weighting is what most affects the efficacy of the GRPO algorithm.

Enhancing Rust Code Generation with GRPO and Compiler-Level Feedback

David Lu
Department of Computer Science
Stanford University
davidlu7@stanford.edu

Cees Armstrong
Department of Mathematics
Stanford University
ceesa@stanford.edu

Miguel Villanueva
Department of Computer Science
Stanford University
miggyjv@stanford.edu

Abstract

Existing RL pipelines for code treat the entire "compile + test" loop as a single Bernoulli signal, this leaves the optimiser blind to why a program fails. In our project, we aim to improve quality of code generation for low compute, strongly-typed languages – in this case, Rust – where the LLM can learn from the diagnostics emitted by the detailed compilers. First, we construct our dataset of prompts by translating AceCoder python prompts into Rust; then, finetune a Qwen2.5-Coder.0.5B model using Group Relative Policy Optimisation (GRPO), a critic-free variant of Proximy Policy Optimisation (PPO) that generates the baseline from other completions of the prompt. Therefore, the algorithm requires no neural reward model and relies solely on formulated reward rubrics. Our reward signals are comprised of compiler-aware signals such as build success, Clippy clean, unit-test passes and assert coverage, which are all gleaned at compile time. Using just a single A100 and one training epoch, the GRPO-tuned model surpasses its refeerence baseline on every metric. It is evident how the 4-signal rubric generates increased Build and Clippy pass rates but fails to generate genuine test success (0.2%) due to reward hacking. In contrast the 7-signal reward rubric, improved with intrinsic and structural reward signals, creates genuine test success (7.20%). Our results show that compiler diagnostics can greatly augment low-resource GRPO code generation and that reward rubric design is essential for quality.

1 Introduction

Training language models (LMs) to produce executable code often suffers from two general obstacles: sparse rewards and opaque credit assignment. Most reinforcement learning pipelines for code, such as CodeRL (Le et al., 2022), AlphaCode-RL (Li et al., 2022), and StepCoder (Dou et al., 2024), treat the entire "compile + test" loop as a single Bernoulli signal. Because that feedback arrives only after many tokens, the optimiser never learns why a program fails (borrow-checker error, type mismatch, unsatisfied assertion), which could often lead to slow convergence/large GPU costs and policies that might overfit the unit test quirks.

Thus, we will be studying Group Relative Policy Optimisation (GRPO) (Shao et al., 2024) in the context of *Rust* code generation.

We choose *Rust* because of its compelling foundation for evaluating code generation due to its stringent syntactic and semantic requirements. The language enforces strict compiler checks at compile-time, rejecting programs that violate safety, ownership, or type constraints, even in relatively minor ways. In addition, Rust incorporates an integrated linter, Clippy, which applies a comprehensive set of lint rules aimed at enforcing stylistic consistency, identifying common correctness pitfalls, and flagging potential performance issues. This combination of strict compilation guarantees and rigorous linting makes Rust particularly well-suited for assessing large language models in code generation tasks, as it provides a high-fidelity signal of code quality and forces code to adhere to best practices.

2 Related Work

The dominant body of program synthesis RL research has been focused on Python datasets. CODERL couples a CodeT5 backbone with PPO but uses a *single* reward (the fraction of tests passed) so the agent receives no signal unless a program compiles and at least one test succeeds (Le et al., 2022). RLTF adds multi-granularity "block" penalties and a compile-bonus, yet still reduces the full compilation pipeline to a scalar before each gradient step (Liu et al., 2023).STEPCODER breaks long problems into a curriculum of shorter sub-tasks, but every sub-task is judged only on final execution outcome and is implemented, again, in Python (Dou et al., 2024). DeepMind's ALPHACODE forgoes RL entirely, ranking thousands of sampled programs by binary pass/fail and thus trading massive compute for sparse reward (Li et al., 2022). Despite these advances, Rust remains relatively absent, primarily because no standard Rust benchmark with executable tests has been available.

A separate line of work examines dense or process-level feedback. Process-supervised RL for code trains a reward model over intermediate reasoning traces and surpasses outcome-only baselines, hinting that richer signals can accelerate learning (Ye et al., 2025). Intrinsic reward shaping with LLM generated rewards has improved learning speed in simulated control environments, again demonstrating the value of dense feedback (Zhang et al., 2023). However, the domain of these studies are different and none of these studies, leverage compiler error codes, lint warnings, or coverage statistics, which are natural, fine-grained signals that arise "for free" during compilation.

We see that unlike Python, Rust ships a strict borrow checker and a first-class build tool (`cargo`). The compiler has richly typed diagnostics (e.g. error `E0382` for "borrow of moved value") that precisely identify failure modes. Clippy exposes $750+$ lints that quantify code quality (Rust Project Development Team, 2025). Tarpaulin measures line coverage in one command (McCorkendale, 2025). These signals are machine verifiable and arrive during compilation, making Rust a natural playground for dense reward shaping. Yet a recent survey of LLM code generators shows almost no results reported for Rust. Early work from Sourcegraph fine-tunes a proprietary model on Rust but does so supervised and without reinforcement feedback, highlighting both the promise and the gap (Liu et al., 2024).

GRPO was introduced in DEEPSEEKMATH to stabilise PPO by normalising rewards across a group of sampled trajectories and has since been extended to reasoning-only self-play in DeepSeek-R1 (DeepSeek-AI et al., 2025). Here, we hope to extend GRPO to the realm of Rust code synthesis. Namely, we aim to fill these gaps by i) synthesising a 12,000 example Rust benchmark via LLM translation of Python tasks with tests; ii) establishing the first GRPO baseline on Rust with the binary compile + test signal adapted to rust; and iii) layering a compiler aware, dense-reward vector that leverages the `rustc` errors, Clippy lints, and coverage metrics. In doing so, we hope to explore whether small coder LMs can match larger counterparts when armed with richer, automatically obtainable feedback and whether dense rewards through the Rust compiler can improve reward efficiency.

3 Method

We use the Qwen2.5-Coder-0.5B model (Yang et al., 2024) and finetune it using GRPO. The Qwen2.5-Coder-0.5B model has been both pre-trained and fine-tuned on code-specific corpora, making it more suited to coding despite its relatively small size. This model serves as the baseline for all subsequent experiments.

GRPO is a critic-free adaptation of Proximal Policy Optimization (PPO) that eliminates the need for a value model, thereby significantly reducing computational overhead. Instead of training a separate

LLM-based critic, GRPO estimates the baseline reward using alternative completions of the same prompt. This design choice makes Reinforcement Learning from Human Feedback (RLHF) more scalable to large language models, cutting GPU memory usage by approximately half.

In our setup, we substitute human-provided feedback with fully automated reward signals derived from the Rust compiler. GRPO generates multiple completions per prompt, assigns a reward to each using task-specific rubrics, and computes advantages by normalizing these rewards relative to the batch mean and standard deviation. A KL-divergence term is added to ensure the updated policy remains close to the reference model.

Figure 1 displays the process for training with GRPO.

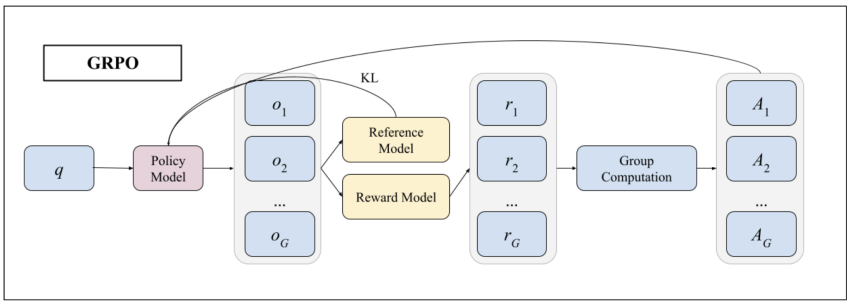


Figure 1: GRPO Process Flow Chart

Concretely, let N be the number of prompts in a batch and K the number of completions generated per prompt. Denote by r_{ij} the raw scalar reward for the jth completion of prompt i. We compute the batch mean μ and standard deviation σ over all NK rewards, then normalize:

$$\hat{r}_{ij} = \frac{r_{ij} - \mu}{\sigma}.$$

These normalized rewards primarily act as the advantages in the clipped surrogate objective

$$L(\theta) = \mathbb{E}_{i,j}\Big[\min\big(\rho_{ij}(\theta)\,\hat{r}_{ij},\ \mathrm{clip}(\rho_{ij}(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{r}_{ij}\big)\Big] \;-\; \beta\,\mathrm{KL}\big[\pi_{\mathrm{old}} \,\|\, \pi_\theta\big],$$

where $\rho_{ij}(\theta) = \pi_\theta(a_{ij}\,|\,s_i)/\pi_{\mathrm{old}}(a_{ij}\,|\,s_i)$.

In general, another important task was selecting effective rewards to balance between *completeness* and *simplicity*. We must cover the core goals, for example successful compilation (build/clippy) and functional correctness (tests passing), while avoiding redundant or conflicting signals. For example, overly granular rewards can pull the model in competing directions, slowing convergence. To give an idea of how we started to approach this, we:

- Start with the minimal extrinsic signals (build, clippy, test).
- Add structural rewards (e.g. code-block presence) only to enforce necessary constraints.
- Introduce intrinsic rewards (assert coverage, non-empty test body) when evaluation loopholes appear.
- Keep the total number of signals small (4–7) so that each gradient step focuses on the most impactful aspects of code quality.

4 Experimental Setup

4.1 Data

We use the AceCoder-87K (Zeng et al., 2025) dataset (with Python prompts) and pre-process it in three ways:

1. Extraction: We pull 15,000 prompts out which we will use for the training and evaluation of our model

2. Translation: We translate all these prompts from Python using GPT `o4-mini` into Rust specific problem statements using a prompt that accounts for Rust's ownership model, type system and syntax.
3. Data Splitting: We split the data into a training, and evaluation set. For our purposes, we save 500 examples for evaluation and train with the remaining 14500.

4.2 Experimental Details

We select the following reward signals for our reward rubric. These are used to govern rewards for each training step within our single Epoch of GRPO. We select the following metrics and their corresponding rewards pts to not only reward passing build, clippy and test functions but also to write well-structured Rust code that robustly tests itself.

Metric	Pts	Description
Build Pass	1.0	`cargo build` succeeds
Clippy Pass	1.0	`cargo clippy` (`-D warnings`) succeeds
Test Pass	2.0	`cargo test` succeeds
Rust Code Block	0.5	Contains a `rust` code fence
Cfg Test Rewards	0.5	Contains a `#[cfg(test)]` function
Assert Coverage	1.0	Increments by 0.25 for every `assert!` (max 4)
Non-empty Body	1.0	≥ 3 non-comment, non-blank lines in tests

Table 1: Reward components used to score each generated Rust snippet.

We run two main fine-tuning experiments:

1. 4-signal Model: We use the first four reward signals, covering Build Pass, Clippy Pass, Test Pass and *Rust* Code Block.
2. 7-signal Model: We use all 7 reward signals, focussing on generating well-structured *rust* code that builds in a wide variety of test cases to improve testing capacity. Both Cfg Test Rewards and Assert Coverage are features of well-structured *rust* code that generates stringent tests to test itself on.

4.3 Finetuning setup

We fine tune `Qwen/Qwen2.5-Coder-0.5B-Instruct` via LoRA (rank $r = 16$, $\alpha = 64$, dropout $= 0.05$) on a single NVIDIA A100 40GB GPU. We set per-device batch size = 4, generate 4 completions per prompt, and accumulate gradients over 1 step. We train for one epoch (roughly 15 hrs), saving checkpoints every 500 steps (keeping the 3 most recent).

4.4 Evaluation Methodology

We evaluate model quality of the fine-tuned model vs the baseline in two ways:

1. Testing on the Evaluation Set: We test the LLM's generated rust code on 500 prompts to gauge its success in passing build, clippy and test functions.
2. Qualitative Evaluation: We go through individual prompts to see their code structure and code quality. This allows us to examine how code generated by the different models differs in quality.

In an earlier version we awarded the full 2.0 points for "Test Pass" whenever `cargo test` exited successfully, even if there were no tests were defined. However, once we qualitatively examined the outputs and the rewards we realized that the LLM had learned to exploit this loophole which we will elaborate more on below. To close that loophole, we now require each snippet to include a real `#[cfg(test)]` module with at least one `assert!()` macro before granting any Test Pass credit.

5 Results

We get the following results when running our models on the 500 prompt evaluation set.

Table 2: Evaluation Results with Baseline

Metric	Count / 500	Percentage
Build Pass Rate	124 / 500	24.80%
Clippy Pass Rate	122 / 500	24.40%
Test Complete & Pass Rate	7 / 500	1.40%
Full Pass Rate (all 3)	7 / 500	1.40%

Table 3: Evaluation Results with 4 Reward Signals

Metric	Count / 500	Percentage
Build Pass Rate	182 / 500	36.40%
Clippy Pass Rate	180 / 500	36.00%
Test Complete & Pass Rate	1 / 500	0.20%
Full Pass Rate (all 3)	1 / 500	0.20%

Table 4: Evaluation Results with 7 Reward Signals

Metric	Count / 500	Percentage
Build Pass Rate	153 / 500	30.60%
Clippy Pass Rate	152 / 500	30.40%
Test Complete & Pass Rate	36 / 500	7.20%
Full Pass Rate (all 3)	36 / 500	7.20%

Breaking down the reward signals for each model, we can see how our 4- and 7-reward signals differ in training reward during the single epoch. We can see this from figures 2 and 3. We output each graph as a line-graphs with a running average of n = 70 over the single epoch of training steps.

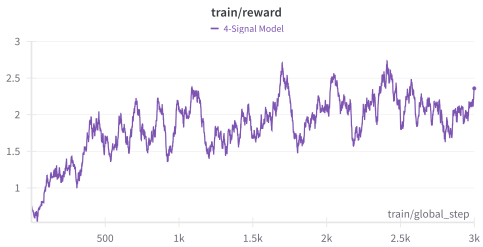


Figure 2: Overall Reward for the 4-signal Model

Figure 3: Overall Reward for the 7-signal Model

The above graphs show overall reward for each function. We also have individual graphs showing how rewards differ for specific instances.

Figure 4: Test Reward Comparison

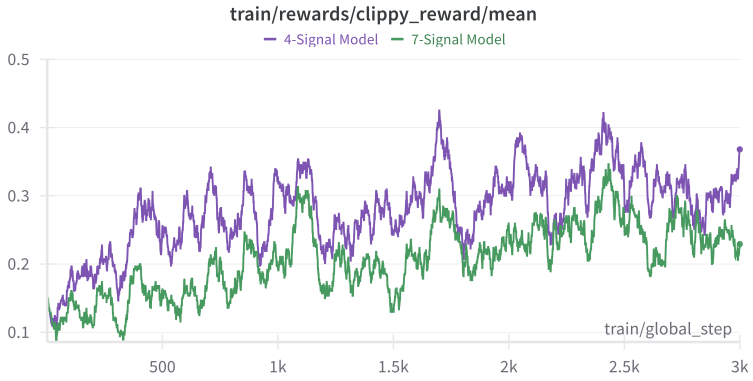


Figure 5: Clippy Reward Comparison

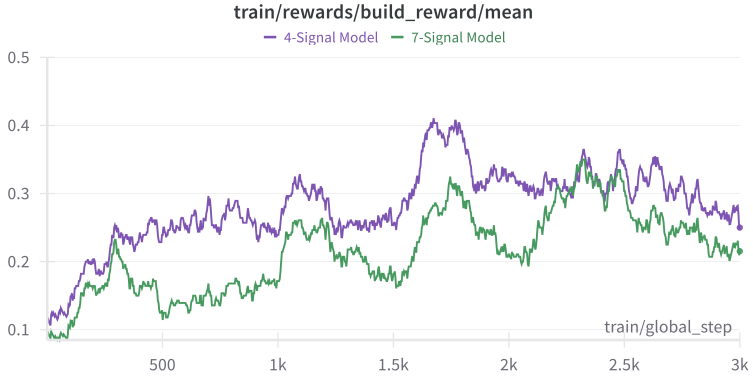


Figure 6: Build Reward Comparison

5.1 Quantitative Evaluation

Table 5: Performance Comparison

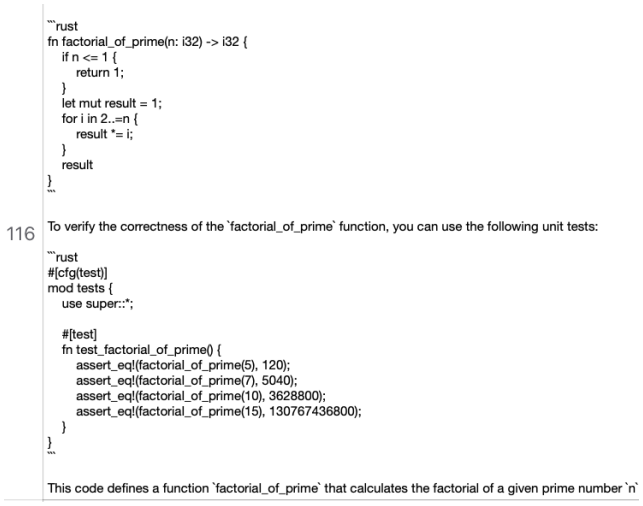
Method	Build Pass Rate	Clippy Pass Rate	Test Pass Rate	Full Pass Rate
Baseline	24.80%	24.40%	1.40%	1.40%
4 Reward Signal	36.40%	36.00%	0.20%	0.20%
7 Reward Signal	30.60%	30.40%	7.20%	7.20%

We see that the 4-reward signal model performs above baseline on build and clippy pass rates, but regresses on test and full pass rates. This can be explained by the poor reward construction. Even though we gave the LLM a very specific system prompt detailing behavior and format of response, such as writing all code and tests in one rust block and to use assert statements, we note that it starts to deviate from it and exploit the rewards. We see that it places the function in the first ““‘rust““‘ fence (which we parse) and dumps ad hoc tests into a separate fence, thereby earning build and Clippy rewards while evading our Test Pass criterion. Due to this, we see that the 4 reward signal not only learns how to write code that is more compilable and clean but it starts to write code that is exploiting Rust's test component by not including tests in the first code block. Thus, under our Test Pass rate which also checks for whether tests exists or not, it completely collapses and fails as the LLM is now purposefully trying to not write tests.

The 7-reward signal in contrast has a lower build and clippy pass rate performance in comparison to the 4-reward model, but improves on test pass rate significantly, beating the baseline by $\sim$5x and the 4 reward signal model by $\sim$35x. This demonstrates that a robust focus on building better structured test cases improves the test pass rates, but that optimising for any specific pass rate has consequences in other places, and that thus building a reward model is not simply a matter of increasing the number of reward signals improving performance across the board, but instead of improving the performance in certain places while decreasing performance in others. However, this might also be caused by the fact that since we are actually writing more complicated tests this is reducing the pass rates of build and clippy naturally.

5.2 Qualitative Analysis

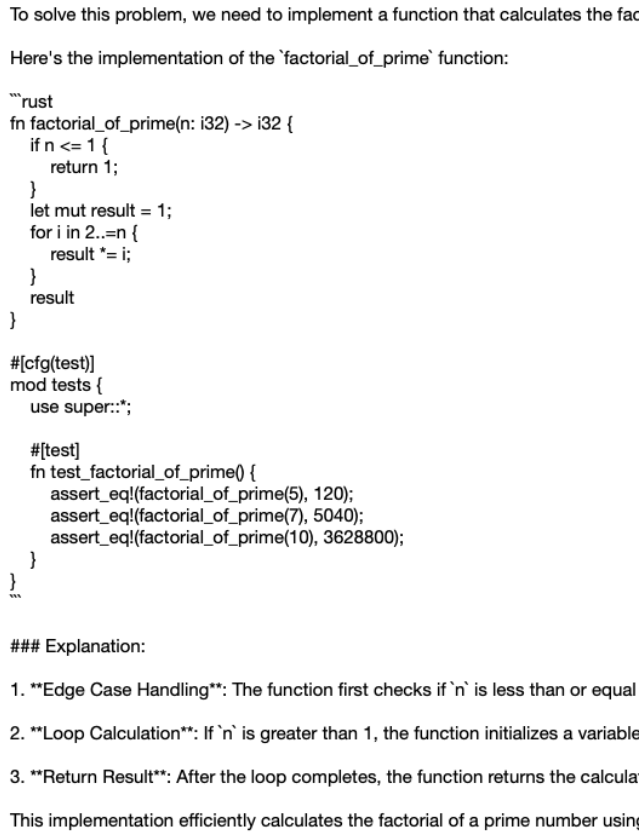
Cheating Under the 4-Signal Scheme. Below in figure 7, we can see how the LLM learned to "answer" our prompt by emitting two separate ‘‘‘`rust`‘‘‘ fences, one containing the function implementation and another containing ad-hoc tests, instead of a single, well-structured `#[cfg(test)]` module. This satisfied the code-block reward and superficially followed our prompt, yet produced no genuine test module. As a result, the model maximized build- and Clippy-pass rewards while also increasing the rewards for passing Test, but at the expense of any authentic test passes.



````
```rust
fn factorial_of_prime(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    }
    let mut result = 1;
    for i in 2..=n {
        result *= i;
    }
    result
}
```

To verify the correctness of the `factorial_of_prime` function, you can use the following unit tests:

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_factorial_of_prime() {
        assert_eq!(factorial_of_prime(5), 120);
        assert_eq!(factorial_of_prime(7), 5040);
        assert_eq!(factorial_of_prime(10), 3628800);
        assert_eq!(factorial_of_prime(15), 130767436800);
    }
}
```

This code defines a function `factorial_of_prime` that calculates the factorial of a given prime number `n`
````

Figure 7: Sample complete for 4_signal.png

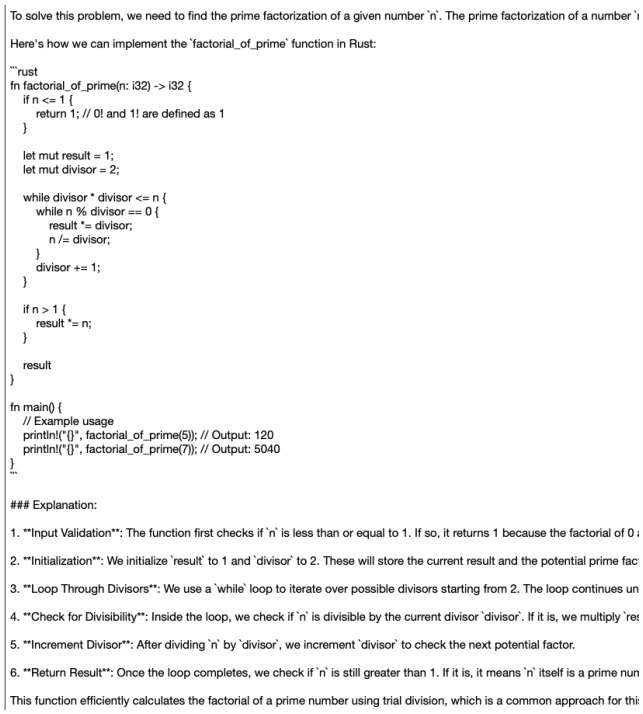
Closing the Loophole with 7 Signals. By contrast, the 7-signal variant explicitly rewards the presence of a `#[cfg(test)]` block and counts distinct `assert!` statements. These additional signals close the loophole: now the model must include and pass real tests to earn any test credit. Once we tightened our evaluation to require one `#[cfg(test)]` module with at least one `assert!()`, the 4-signal model's genuine test-pass rate collapsed to near zero, while the 7-signal model—in spite of a slight drop in build/Clippy throughput—achieved substantial, authentic test-pass gains. We can see a sample output of the 7-signal model in figure 8.



````
To solve this problem, we need to implement a function that calculates the fac

Here's the implementation of the `factorial_of_prime` function:

```rust
fn factorial_of_prime(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    }
    let mut result = 1;
    for i in 2..=n {
        result *= i;
    }
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_factorial_of_prime() {
        assert_eq!(factorial_of_prime(5), 120);
        assert_eq!(factorial_of_prime(7), 5040);
        assert_eq!(factorial_of_prime(10), 3628800);
    }
}
```

### Explanation:

1. **Edge Case Handling**: The function first checks if `n` is less than or equal

2. **Loop Calculation**: If `n` is greater than 1, the function initializes a variable

3. **Return Result**: After the loop completes, the function returns the calcula

This implementation efficiently calculates the factorial of a prime number using
````

Figure 8: Sample complete for 7_signal.png

As a reference point, we have a sample completion for the baseline in figure 9.



To solve this problem, we need to find the prime factorization of a given number `n`. The prime factorization of a number `n

Here's how we can implement the `factorial_of_prime` function in Rust:

```rust
fn factorial_of_prime(n: i32) -> i32 {
    if n <= 1 {
        return 1; // 0! and 1! are defined as 1
    }

    let mut result = 1;
    let mut divisor = 2;

    while divisor * divisor <= n {
        while n % divisor == 0 {
            result *= divisor;
            n /= divisor;
        }
        divisor += 1;
    }

    if n > 1 {
        result *= n;
    }

    result
}

fn main() {
    // Example usage
    println!("{}", factorial_of_prime(5)); // Output: 120
    println!("{}", factorial_of_prime(7)); // Output: 5040
}
```

Explanation:

1. **Input Validation**: The function first checks if `n` is less than or equal to 1. If so, it returns 1 because the factorial of 0 a

2. **Initialization**: We initialize `result` to 1 and `divisor` to 2. These will store the current result and the potential prime fac

3. **Loop Through Divisors**: We use a `while` loop to iterate over possible divisors starting from 2. The loop continues un

4. **Check for Divisibility**: Inside the loop, we check if `n` is divisible by the current divisor `divisor`. If it is, we multiply `res

5. **Increment Divisor**: After dividing `n` by `divisor`, we increment `divisor` to check the next potential factor.

6. **Return Result**: Once the loop completes, we check if `n` is still greater than 1. If it is, it means `n` itself is a prime nun

This function efficiently calculates the factorial of a prime number using trial division, which is a common approach for thi

Figure 9: Sample complete for base.png

6 Discussion

We see that reward signals are central to the effectiveness of GRPO, but their design must be approached with care to avoid reward hacking, in which the model optimizes for the reward function in unintended ways. However, when appropriately selected, well-aligned reward functions can drive substantial gains across all dimensions of *Rust* code quality.

We were limited by our compute, but further work ought to be done running experiments both with different mixtures of reward signals (venturing beyond our chosen 7) and with different reward values per reward signal — for example, passing the test function could be rewarded more highly. Training over several epochs would also be elucidating to see at what point GRPO fine-tuning's effectiveness levels off. Research of this type would help us to further understand how the reward signals chosen impact our models and how we could this best improve our models.

7 Conclusion

In our project, by using a light 0.5-b Qwen2.5-Coder and critic-free GRPO, we demonstrate that policy-gradient fine-tuning is possible on a single A100 (a tiny GPU budget for reinforcement learning) for low-resource code generation. *Rust*, as a language with a heavy compile chain, provided more detailed error messages and stricter compilation rules which enabled us to use compile-aware reward signals. We developed seven reward signals that combine `cargo build`, `clippy`, unit-test success, and structural cues (presence of a `#[cfg(test)]` block, assert coverage). These signals are automatically extracted and incur no labelling cost. After we fine-tuned that 0.5-b coder LM with GRPO and our dense, compiler-aware rewards, the RL-tuned model beat its own baseline version on all of the reward scores. It is clear that reward design, more than model size, decided performance. Reward signals are essential for GRPO to work, but they need to be carefully chosen to make sure that reward hacking doesn't occur — it is necessary to examine the outputs and not rewards to make sure

that the rewards holistically improve quality of code generation. Adding incentives in the 7-reward signal model compared to the 4-reward signal model ensured that the tests were structurally sound and not trivial, established authentic unit-test success. Our results emphasise the practical value of treating compiler diagnostics as intrinsic, wide-ranging feedback for reinforcement learning in statically and strongly typed coding languages. By relying exclusively on deterministic compiler signals and group statistics, our loop follows the concept behind GRPO-Zero – achieving reinforcement learning from fully automated feedback without any neural reward model.
Our work is limited by two main factors. Firstly, all our experiments were run for a single epoch on a single A100. As a result, the true ceiling and efficiency of GRPO is left uncertain. Next, our benchmark is artificial and comes from AceCoder which does not necessarily reflect real-world Rust projects. Further exploration into real industrial codebases, and even other strongly typed coding languages, would assist in testing the generality of our outputs and findings.
A clear next step for us would be to test the scalability of our models and do more investigation into different reward signals and rubrics — this includes extending training across multiple epochs and model sizes to better understand when performance flatlines.

8 Team Contributions

- **Cees:** Cees contributed to the data pipeline in addition to the reward code.
- **David:** David mainly contributed to the reward code and training pipeline.
- **Miguel:** Miguel focused on the training pipeline in addition to the eval code.

Changes from Proposal Donec et nisl at wisi luctus bibendum. Nam interdum tellus ac libero. Sed sem justo, laoreet vitae, fringilla at, adipiscing ut, nibh.

References

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, and et al. Junxiao Song. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

Shihan Dou, Yan Liu, Haoxiang Jia, Limao Xiong, Enyu Zhou, Wei Shen, Junjie Shan, Caishuang Huang, Xiao Wang, Xiaoran Fan, Zhiheng Xi, Yuhao Zhou, Tao Ji, Rui Zheng, Qi Zhang, Xuanjing Huang, and Tao Gui. 2024. StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback. arXiv:2402.01391 [cs.SE] https://arxiv.org/abs/2402.01391

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. arXiv:2207.01780 [cs.LG] https://arxiv.org/abs/2207.01780

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. , 1092–1097 pages. https://doi.org/10.1126/science.abq1158

Beyang Liu, Rishabh Mehrotra, and Hitesh Sagtani. 2024. Enhancing Code Completion for Rust in Cody. https://sourcegraph.com/blog/enhancing-code-completion-for-rust-in-cody. Sourcegraph Blog.

Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. 2023. RLTF: Reinforcement Learning from Unit Test Feedback. arXiv:2307.04349 [cs.AI] https://arxiv.org/abs/2307.04349

Steven McCorkendale. 2025. Tarpaulin: A Code Coverage Tool for Rust Projects. https://github.com/xd009642/tarpaulin. GitHub repository, accessed 22 Apr 2025.

Rust Project Development Team. 2025. *Clippy: A Collection of Lints to Catch Common Mistakes and Improve Your Rust Code*. https://doc.rust-lang.org/clippy/ Online documentation, accessed 22 Apr 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] https://arxiv.org/abs/2402.03300

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

Yufan Ye, Ting Zhang, Wenbin Jiang, and Hua Huang. 2025. Process-Supervised Reinforcement Learning for Code Generation. arXiv:2502.01715 [cs.SE] https://arxiv.org/abs/2502.01715

Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhu Chen. 2025. AceCoder: Acing Coder RL via Automated Test-Case Synthesis. *arXiv preprint arXiv:2207.01780* (2025).

Alex Zhang, Yao Xiao, and Paul Liang. 2023. A Simple Framework for Intrinsic Reward-Shaping for RL using LLMs. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. https://alexzhang13.github.io/assets/pdfs/Reward_Shaping_LLM.pdf